

Replicated Counter Service Test Report

Student: Ravil Gubaidullin

Repository: <https://github.com/rava1506/dcc-assignment2>

1. Requirements Under Test

The project implements a distributed counter service using Python and gRPC. Clients can increment named counters and retrieve their current values. The implementation includes idempotency keys, bounded retries, thread synchronization, Lamport logical clocks, and a three-replica deployment with majority-acknowledged writes.

The purpose of testing is to verify normal operation, concurrent access, duplicate suppression, timeout handling, and behavior when replicas become unavailable. Performance testing compares single-replica requests with the three-replica quorum path.

Assignment requirement	Verification
A2 - Increment applies the requested delta	<code>test_increment_applies_delta</code>
A2 - Duplicate requests do not repeat the mutation	<code>test_duplicate_key_not_reapplied</code>
A2 - Retries after timeouts are safe	<code>test_retry_after_timeout_is_safe</code>
A3 - Concurrent increments remain correct	<code>test_concurrent_increments_exact</code>
B1 - Lamport clocks follow the update rules	<code>test_lamport_rules</code>
B1 - Logical time is attached to requests and replies	<code>test_protocol_lamport_round_trip</code> and <code>event logs</code>
C1 - Two acknowledgements permit a commit	<code>test_majority_commit_two_acks</code>
C1 - One acknowledgement does not permit a commit	<code>test_no_commit_below_majority</code>
C1 - Fully delivered writes produce equal replica values	<code>test_replicas_converge</code>

2. Test Environment

Component	Configuration
Operating system	Windows-11-10.0.26200-SP0
Python version	3.13.0 (tags/v3.13.0:60403a5, Oct 7 2024, 09:38:07) [MSC v.1941 64 bit (AMD64)]
Computer	AMD64
Processor	AMD64 Family 25 Model 68 Stepping 1, AuthenticAMD
RAM	24GB
Communication	gRPC over the local loopback network
Server concurrency	Eight worker threads per replica
Replication	Three independent replicas
Commit threshold	Two acknowledgements from different replicas
Default RPC deadline	Two seconds
Retry policy	Initial attempt and up to three retries

Retry delays	0.2, 0.4, and 0.8 seconds
--------------	---------------------------

The project uses `grpcio`, `grpcio-tools`, and `pytest`. The installed package versions are included in Appendix B.

Automated tests create fresh server states on ephemeral ports and clean up their resources afterwards. The crash-injection test and performance benchmark start replicas as separate processes. Selected failure tests use shorter deadlines to produce faults deterministically; the default client configuration remains unchanged.

3. Test Cases and Results

The test suite was executed using:

```
python -m pytest tests -v -s --junitxml=logs/test_results.xml
```

```
(.venv) PS C:\Users\RTX\Desktop\dcc-assignment2> python -m pytest tests -v -s --junitxml=logs/test_results.xml
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\RTX\Desktop\dcc-assignment2\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\RTX\Desktop\dcc-assignment2
collected 15 items

tests/test_counter.py::test_increment_applies_delta PASSED
tests/test_counter.py::test_duplicate_key_not_reapplied PASSED
tests/test_counter.py::test_get_missing_counter PASSED
tests/test_counter.py::test_concurrent_increments_exact PASSED
tests/test_counter.py::test_retry_after_timeout_is_safe TIMEOUT: deadline observed; same-key retry succeeded; value=7; applies=1
PASSED
tests/test_counter.py::test_majority_commit_two_acks PASSED
tests/test_counter.py::test_no_commit_below_majority PASSED
tests/test_counter.py::test_replicas_converge PASSED
tests/test_counter.py::test_lamport_rules PASSED
tests/test_counter.py::test_protocol_lamport_round_trip PASSED
tests/test_counter.py::test_counter_isolation PASSED
tests/test_counter.py::test_key_conflict_is_rejected PASSED
tests/test_failures.py::test_replica_crash_mid_request CRASH: killed after RECV; committed=True; acks=2; values=[1,1]
PASSED
tests/test_failures.py::test_request_duplication DUPLICATE: second reply was_duplicate=True; value=5
PASSED
tests/test_failures.py::test_induced_timeout_with_retry TIMEOUT: deadline observed; same-key retry succeeded; value=7; applies=1
PASSED

----- generated xml file: C:\Users\RTX\Desktop\dcc-assignment2\logs\test_results.xml -----
===== 15 passed in 5.53s =====
```

The original execution evidence is stored in `logs/test_results.xml` and `logs/pytest_results.txt`.

RPC Semantics and Synchronization

The implementation provides at-most-once mutation per idempotency key on each replica while its deduplication state remains available. The client generates one key per logical operation and reuses it across retry attempts.

Each replica checks the deduplication store and modifies its counter under the same `threading.Lock`. This prevents concurrent requests with the same key from both applying the mutation. A duplicate returns the original operation result, which may differ from the counter’s current value after subsequent writes.

These properties are evaluated by `test_duplicate_key_not_reapplied`, `test_retry_after_timeout_is_safe`, and `test_concurrent_increments_exact`.

4. Failure-Injection Results

Three failure scenarios were automated. Each scenario checks an observable invariant rather than relying only on a printed success message.

Scenario	Injected fault	Expected invariant	Actual result and evidence
Replica crash during a request	Kill a replica after its received-request marker appears	The two surviving replicas acknowledge the write; the client reports commit without an unhandled RPC exception	The replica was killed after receiving the request. The client reported committed=True with two acknowledgements. Both surviving replicas returned value 1. Evidence: test_replica_crash_mid_request - PASS; logs/failure-crash.log.
Duplicate delivery	Send the same operation with the same key twice	The second reply indicates a duplicate; the delta is applied once	The second response returned was_duplicate=True. The counter remained at 5, confirming that the delta was applied only once. Evidence: test_request_duplication - PASS; logs/failure-duplicate.log.
Timeout followed by retry	Delay the first reply after applying the mutation	The client observes a timeout, retries with the same key, and receives the stored result	The client observed DEADLINE_EXCEEDED and retried using the same idempotency key. The retry succeeded with was_duplicate=True; the counter remained at 7 with exactly one mutation. Evidence: test_induced_timeout_with_retry - PASS; logs/failure-timeout.log.

Crash evidence:

```
logs > failure-crash.log
You, 13 minutes ago | author (You)
1 READY 127.0.0.1:52249
2 [replica-2] RECV Increment(counter=x, delta=1) L=4 (received L=3)
3
4 KILLED after RECV; committed=True; acks=2; surviving values=[1,1]
5 |
```

Duplicate-delivery evidence:

```

logs > failure-duplicate.log
You, 3 hours ago | author (You)
1  [replica-A] RECV Increment(counter=x, delta=5) L=2 (received L=1)
2  [replica-A] APPLY counter=x -> 5 L=3
3  [replica-A] SEND IncrementReply(new_value=5) L=4
4  [replica-A] RECV Increment(counter=x, delta=5) L=7 (received L=6)
5  [replica-A] DUP counter=x -> 5 L=8
6  [replica-A] SEND IncrementReply(new_value=5) L=9
7  [replica-A] RECV Get(counter=x) L=12 (received L=11)
8  [replica-A] READ counter=x -> 5, found=True L=13
9  [replica-A] SEND GetReply(value=5, found=True) L=14
10

```

Timeout evidence:

```

logs > failure-timeout.log
You, 3 hours ago | author (You)
1  [replica-A] RECV Increment(counter=x, delta=7) L=2 (received L=1)
2  [replica-A] APPLY counter=x -> 7 L=3
3  [replica-A] RECV Increment(counter=x, delta=7) L=5 (received L=4)
4  [replica-A] DUP counter=x -> 7 L=6
5  [replica-A] SEND IncrementReply(new_value=7) L=7
6  [replica-A] RECV Get(counter=x) L=10 (received L=9)
7  [replica-A] READ counter=x -> 7, found=True L=11
8  [replica-A] SEND GetReply(value=7, found=True) L=12
9

```

Partial Application Below Majority

A write that receives fewer than two acknowledgements is reported as uncommitted. However, a surviving replica may already have applied it. The implementation does not roll back that local mutation. This behavior is explicitly checked by `test_no_commit_below_majority`.

A full consensus protocol would use a commonly agreed log and commit rules to determine which entries may be applied and exposed as committed state. Independent local mutations followed by acknowledgement counting do not provide the same guarantee.

Single-Replica Read Risk

A `Get` request may read from one replica. That replica can be missing writes or contain a partially applied operation, so the returned value is not guaranteed to represent a single globally committed state. The implementation does not provide linearizable reads.

5. Lamport Ordering Analysis

The B2 scenario starts three independent client processes. Client-1 increments counter `x` twice, client-2 increments counter `y` twice, and client-3 performs only `Get` operations.

Each participant maintains its own Lamport clock. Before a local event, its clock increases by one. On receipt of a message, it updates its clock using:

$$L = \max(\text{local_time}, \text{received_time}) + 1$$

The selected event trace is stored in `logs/event_excerpt.log` and included in Appendix A. The excerpt groups events by participant and preserves their local order; its line order is not a global physical-time ordering.

Causally Ordered Pair 1

Client-1's first `SEND Increment(x)` at **L=1** happens-before the corresponding replica-A `RECV` at **L=10**.

This relationship follows from the message edge between sending and receiving the same request.

Causally Ordered Pair 2

Client-1's first `RECV IncrementReply` at **L=13** happens-before its second `SEND Increment(x)` at **L=14**.

This relationship follows from local program order: the client receives the first reply before issuing its next operation. The second request's reception at replica-A extends this chain through another message edge.

Concurrent Events

Client-1's first `SEND` has **L=1**, while client-2's first `SEND` has **L=3**. Client-2 performs two local initialization events before its first send.

The scenario barrier prevents transmission of the first requests until all clients have logged their first `SEND`. Neither selected event is preceded by an incoming reply, and there is no happened-before chain between the two selected sends. Their different Lamport values therefore do not establish causality or physical order.

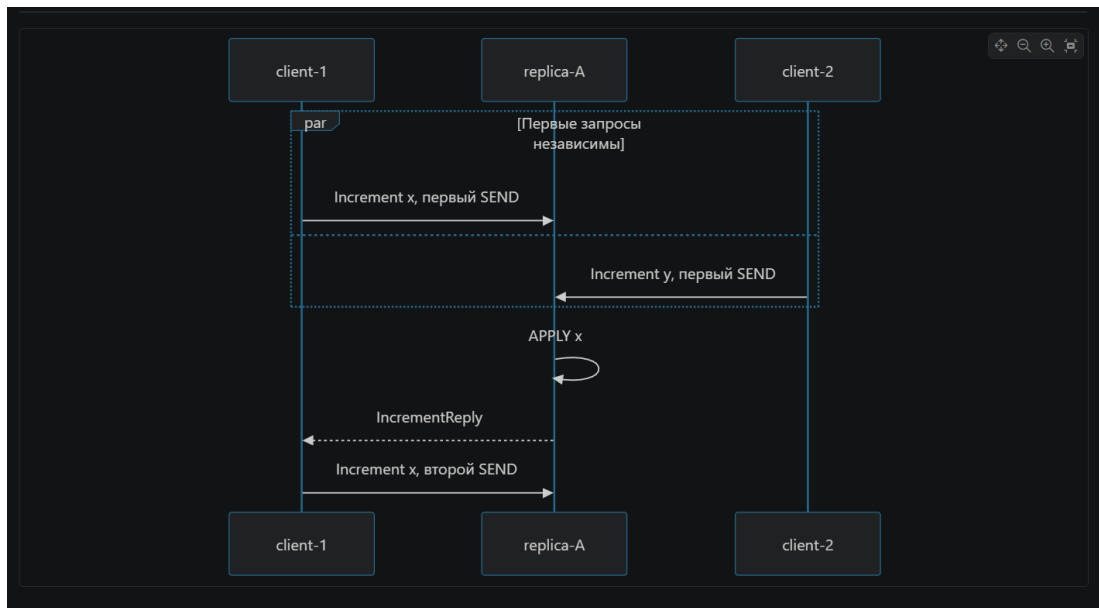


Figure 1. Happened-before diagram for the selected events

Limitation of Lamport Clocks

If event a happens-before event b, then $L(a)$ is smaller than $L(b)$. The converse is not guaranteed: a smaller timestamp alone does not establish a causal relationship. Vector clocks address this limitation by identifying concurrency through incomparable timestamp vectors.

6. Performance Evaluation

The benchmark compares four configurations, with 2,000 measured logical increments per configuration. Each configuration includes 50 warmup operations that are excluded from the measurements. With 16 clients, each client performs 125 measured operations.

Channels are created before measurement and reused. Detailed console event logging is disabled consistently across configurations, while logical-clock instrumentation remains enabled. Replicas run as independent processes over the loopback network.

Latency is measured using `time.perf_counter()` around the full client operation. The quorum client sends requests in parallel but waits for all bounded attempts to finish before evaluating whether at least two replicas acknowledged.

The median is calculated from the recorded samples. The p95 calculation uses the nearest-rank method:

```
p95 = sorted_latencies[ceil(0.95 * n) - 1]
```

The subtraction accounts for Python's zero-based indexing. Raw samples are retained in `logs/latencies.csv`.

Configuration	Median latency (ms)	p95 latency (ms)	Requests
Single replica, 1 client	0.6607500254176557	0.8792000007815659	2,000
Single replica, 16 clients	5.0266500329598784	5.620900017675012	2,000
Quorum (3 replicas), 1 client	1.0172000038437545	1.1501000262796879	2,000
Quorum (3 replicas), 16 clients	8.66729998961091	21.344399952795357	2,000

Interpretation

With one client, the measured median latency was 0.6607500254176557 ms for a single replica and 1.0172000038437545 ms for quorum. The quorum path adds request fan-out, processing on multiple replicas, acknowledgement collection, and client-side scheduling. At 16 concurrent clients, the measured p95 changed from 5.620900017675012 ms to 21.344399952795357 ms, reflecting the combined effects of concurrency and the replication path. The measured quorum p95 is acceptable for this workload, although local loopback results do not establish performance over a production network.

7. Known Limitations

1. **No durable state.** Counter values and deduplication results are stored in memory and are lost when a replica restarts.
2. **No automatic catch-up.** A restarted or disconnected replica does not automatically recover missed operations. Duplicate suppression prevents repeated execution but does not deliver missing writes.
3. **Partial failed writes.** An operation may change one replica even when the client cannot obtain a majority.

4. **Potentially stale reads.** A single-replica `Get` does not guarantee a globally committed or linearizable value.
5. **Unbounded deduplication storage.** The set of remembered operation keys grows without an expiration policy.
6. **Waiting for all attempts.** The quorum client waits for all bounded replica attempts instead of returning immediately after the second acknowledgement.

8. Reflection Questions

1. Why can a client not distinguish a lost request from a lost reply after a timeout?

A timeout can mean that the request was lost, the server is slow, or the mutation completed but its reply was lost. These situations appear the same to the client. Reusing the same idempotency key allows a retry to obtain the stored result if the mutation already occurred, preventing another application while the deduplication state remains available.

2. Why does a smaller Lamport value not imply that an event happened first?

Independent processes may advance their clocks by different amounts. Two concurrent events can therefore have different scalar timestamps. Causality must be established through local program order and matching message edges, rather than by comparing Lamport values alone.

3. Which failures can the three-replica service tolerate, and how does this relate to safety and liveness?

One unavailable replica leaves two replicas able to acknowledge new writes. In the crash-stop model without state-losing restarts, a write acknowledged by two replicas still has at least one surviving copy after one replica crashes.

Two unavailable replicas prevent new majority-acknowledged writes, so write liveness is lost. The client still follows the commit rule by refusing to present a single acknowledgement as a commit. Losing both acknowledging copies can remove a particular write, and in-memory restarts can also lose data. Quorum intersection alone does not establish full consensus safety or linearizable reads.

4. What is the smallest change needed to preserve state across replica restarts?

Counter values and deduplication results should be stored atomically in persistent storage, for example in one SQLite transaction per mutation. Both must survive together to preserve retry safety. Write-ahead logging anticipates durable logging mechanisms used in consensus systems, while recovery of missed writes would additionally require replay or log replication.

Appendix A - Captured Event Trace

```
logs > event_excerpt.log
You, 9 hours ago | author (You)
1 [client-1] SEND Increment(counter=x, delta=1) L=1
2 [client-1] RECV IncrementReply(new_value=1) L=13 (received L=12) You, 9 hours ago * test: capture three-
3 [client-1] SEND Increment(counter=x, delta=1) L=14
4 [client-1] RECV IncrementReply(new_value=2) L=22 (received L=21)
5 [client-2] START scenario initialization L=1
6 [client-2] START counter y selected L=2
7 [client-2] SEND Increment(counter=y, delta=1) L=3
8 [client-2] RECV IncrementReply(new_value=1) L=16 (received L=15)
9 [client-2] SEND Increment(counter=y, delta=1) L=17
10 [client-2] RECV IncrementReply(new_value=2) L=25 (received L=24)
11 [client-3] SEND Get(counter=x) L=1
12 [client-3] RECV GetReply(value=0, found=False) L=5 (received L=4)
13 [client-3] SEND Get(counter=y) L=6
14 [client-3] RECV GetReply(value=0, found=False) L=10 (received L=9)
15 [replica-A] RECV Get(counter=x) L=2 (received L=1)
16 [replica-A] READ counter=x -> 0, found=False L=3
17 [replica-A] SEND GetReply(value=0, found=False) L=4
18 [replica-A] RECV Get(counter=y) L=7 (received L=6)
19 [replica-A] RECV Increment(counter=x, delta=1) L=10 (received L=1)
20 [replica-A] APPLY counter=x -> 1 L=11
21 [replica-A] SEND IncrementReply(new_value=1) L=12
22 [replica-A] RECV Increment(counter=y, delta=1) L=13 (received L=3)
23 [replica-A] APPLY counter=y -> 1 L=14
24 [replica-A] SEND IncrementReply(new_value=1) L=15
25 [replica-A] RECV Increment(counter=x, delta=1) L=19 (received L=14)
26 [replica-A] APPLY counter=x -> 2 L=20
27 [replica-A] SEND IncrementReply(new_value=2) L=21
28 [replica-A] RECV Increment(counter=y, delta=1) L=22 (received L=17)
29 [replica-A] APPLY counter=y -> 2 L=23
30 [replica-A] SEND IncrementReply(new_value=2) L=24
31
```

Appendix B - Installed Package Versions

```
logs > pip-freeze.txt
You, 3 hours ago | author (You)
1 colorama==0.4.6
2 grpcio==1.84.0
3 grpcio-tools==1.84.0
4 iniconfig==2.3.1
5 packaging==26.3
6 pluggy==1.6.0
7 protobuf==7.36.2
8 Pygments==2.21.0
9 pytest==9.1.1
10 setuptools==84.0.0
11 typing_extensions==4.16.0
12
```

Appendix C - Test Execution Evidence

```
logs > pytest_results.txt
13 tests/test_counter.py::test_majority_commit_two_acks PASSED
14 tests/test_counter.py::test_no_commit_below_majority PASSED
15 tests/test_counter.py::test_replicas_converge PASSED
16 tests/test_counter.py::test_lamport_rules PASSED
17 tests/test_counter.py::test_protocol_lamport_round_trip PASSED
18 tests/test_counter.py::test_counter_isolation PASSED
19 tests/test_counter.py::test_key_conflict_is_rejected PASSED
20 tests/test_failures.py::test_replica_crash_mid_request CRASH: killed after RECV; committed=True; acks=2; values=[1,1]
21 PASSED
22 tests/test_failures.py::test_request_duplication DUPLICATE: second reply was_duplicate=True; value=5
23 PASSED
24 tests/test_failures.py::test_induced_timeout_with_retry TIMEOUT: deadline observed; same-key retry succeeded; value=7; applies=1
25 PASSED
26
27 - generated xml file: C:\Users\RTX\Desktop\dcc-assignment2\logs\test_results.xml -
28 ===== 15 passed in 6.69s =====
29
```